Entity Resolution with FAISS + Splink

One system, three conversations

Denis Robert

How to use this deck

Part	Audience	Focus	After the talk
1	C-suite	Value, risk, cost	Decision / review
2	Engineering leadership	Architecture, design, roadmap	Carry it forward
3	Line engineers	Orientation, stack, hands-on	**Go hands-on**

> Line engineers: this orients you. The whitepaper (`docs/entity_resolution_whitepaper.pdf`) is the source of record; the numbers here come from it.

Part 1 · Executive Summary

The 60-second version

The problem

Organisations hold **many records for the same real person**, and they disagree.

- Names differ (e.g., `R. Martinez` vs `Robert Martinez`).
- Addresses change; emails and dates go missing (**30% missing** here).
- Duplicates fragment CRM, risk, and reporting.

Wrong answers are expensive: linking two *different* people is usually far worse than missing two records of the same person.

The solution: two stages

> Fast retrieval + explainable matching.

```
Query ──► [embed] ──► [FAISS top-k]
      ──► [Splink linkage] ──► Match / No match
```

- **Blocking** (FAISS): cheaply finds the few most likely matches.
- **Linkage** (Splink): weighs the evidence per field and returns a probability, not just yes/no.

What it delivers

- **99.5% F1** on the synthetic test corpus (15,000 labelled queries).
- **~7.5 ms/query amortized** *batched* throughput; the honest **online** per-query path is slower (~seconds, dominated by per-query Splink construction).
- **Persisted and reusable**: the index reloads without re-embedding the population.
- **Defensible numbers**: every result reproduces by re-running the code.

Set the risk dial, not a magic number

Splink returns a **probability**; a pair is called a match when that probability is at or above a threshold `τ`. One number encodes the business trade-off.

- **Raise** `τ` → fewer false positives (fewer *wrong* links) at slightly more false negatives.
- **Lower** `τ` → catch more true matches, at more false positives.

The cost-based rule:

```
τ* = C_FP / (C_FP + C_FN)
```

If a **false link** (FP) is 20× more costly than a **missed link** (FN), then `τ* = 20/21 ≈ 0.95` — the model is tuned to the cost of each mistake. (Valid under calibrated probabilities; with labels, sweep `τ` on validation — whitepaper Appendix — Deriving τ.)

> One parameter lets a risk-averse line (fraud/AML) and a recall-first line (marketing) share the same model, each tuned to its own economics.

Cost and scale

- The 50,000-record index is **~87 MB** today.
- Capacity is predictable from RAM;

RAM	Recommended capacity
16 GB	~6.9 M records
32 GB	~13.8 M records
128 GB	~55.1 M records

Around **7–14 M records** (or when you need replicated, multi-region durability) buy a **vector database**; before that the in-process index is the lean choice.

The honest caveats

- Results are measured on **synthetic data**; production needs a **labelled sample of true duplicates** to calibrate and validate.
- **Retraining the match model without also resetting the threshold backfires.** The shipped defaults work best until proper joint calibration. Don't let a team "just retune" today.
- Volume must clear our capacity thresholds before any external-infra spend.

The ask

- Sponsor a short **proof-on-real-data** phase: a representative, labelled sample of the population we must link (with access/privacy).
- We then commit to an F1 on *that* data, not the synthetic one.

Value in one line: near-perfect, explainable identity resolution with batched throughput ~7.5 ms/query, and a path to scale -- but online per-query latency is seconds (Splink rebuild per call), which the paper is honest about and gives a future direction for.

Part 2 · Engineering Leadership

Architecture, measurements, where to invest

Architecture at a glance

- Embeddings are **L2-normalized**, so FAISS inner product = cosine.
- Splink = **Fellegi–Sunter**: per-field weight `w = log2(m/u)`, combined with a match prior, compared to `τ`.
- A store abstraction lets us swap the in-memory index for an **external vector DB** later without changing blocking or linkage.

Where performance comes from

Blocking recall is the ceiling; Splink is where F1 is won or lost.

- On the confusion-matrix query set (15k queries): top-20 blocking recall **98.89%** → end-to-end recall **98.86%** — the linkage stage rejects only 3 of the 9,889 retrieved positives; blocking is the binding constraint.
- A "perfect" blocker would take F1 from **99.31** → **~99.87%**, since 111 positives were missed at top-20; retrieval now has real headroom.
- (Section 7, a different query construction: blocking recall 99.75%; compact raises it to 99.88%.)

> Design implication: improving the embedding/blocking engine is **not** the lever today; linkage configuration is.

The measured levers

Lever	Effect	Verdict
Threshold τ	0.85→0.95: F1 barely moves (99.31→99.35%); recall scarce	Small—retrieval is the lever now
Serialization	single seed: recall 99.88%, F1 99.83%; five-seed mean F1 99.90 vs 99.82 (+0.08)	Yes—small, reproducible
Blocking size k	20→100: +recall, 2.4× Splink time	Diminishing returns
Weaken address	never beat full-strength	No (this data)
Retrain m/u	§8.1: untrained 97.6% vs supervised 95.6%; §8.2 100k: EM at default prior is best (97.9%)	Prior and data-dependent

`τ` is the cost dial (engineering view)

Splink's decision rule: classify as *match* when `P(M|γ) ≥ τ` (range ~[0.5, 0.999]).

- **Raise** `τ` → fewer false positives, at more false negatives.
- **Lower** `τ` → more true matches, at more false positives.

Decision-theoretic setting: `τ* = C_FP / (C_FP + C_FN)` — presumes a calibrated posterior, zero cost for correct decisions, and per-pair decisions (whitepaper Appendix — Deriving τ).

- Measured: `τ` 0.85 → 0.95 moved F1 barely (99.31% → 99.35%; FP 23→6) because recall — not threshold — is the scarce resource.
- With recall headroom, raising `τ` is the cheapest precision lever — the mechanism to encode FP/FN business costs. (Methods: whitepaper **Appendix — Deriving τ**.)

The calibration paradox

Retraining the match model (`m/u`) made results **worse**, not better.

1. **Prior coupling (dominant):** EM's *free* prior (0.0071 vs 0.0001) shifted scores up -> mass false positives in earlier data. **Pin the prior and EM becomes the best variant** on the 100k duplicate benchmark (97.85% F1, 100% precision/specificity).
2. **The residual shrinks with realistic data:** supervised still trails untrained in 8.1 (95.6% vs 97.6%) at the inherited prior/threshold, but EM at the default prior now wins on the duplicate-bearing set -- the residual is data- and prior-dependent.

> EM training generates candidate pairs by exact-equality blocking on two keys: `first_name` and `date_of_birth` (`block_on("first_name")` , `block_on("date_of_birth")`).

> **Rule:** the default config is a coherent bundle (weights + prior + `τ`). Change any part and re-validate **all three** on held-out data.

The fix is a recipe, not a guess: **anchor the prior** (default or blocking-adjusted), fit `m/u` with the prior pinned, then **tune** `τ` **jointly**, with a score-shift guard. Full algorithm: **Appendix — Tuning the Match Prior**.

See paper §8.1 and the joint `τ × prior` table.

Capacity and when to buy infra

- **Memory is the binding constraint** (~1,746 bytes/record).
- Move to an **external vector DB** around **7–14 M records**, or when you need shared/multi-region **durability and replication**, or beyond **~50 M** as a rule of thumb.
- The persisted index reloads **without re-embedding**, so redeploy and multi-consumer serving are cheap.

Operational readiness

- Persisted, reload-friendly store with `update` / `delete`; external-store interface is the vector-DB swap point.
- **Reproducible:** every experiment maps to a script; scripts accept `--input`.

Roadmap

1. Invest in **linkage calibration** (labelled pairs + joint τ/prior tuning).
2. Do **not** spend on blocking-engine research at this scale. (Future: canopy/embedding-blocked `m/u` training — §Further Research — but only where blocking recall is the binding constraint.)
3. Add a **CI gate** locking F1 on a held-out labelled set.
4. **Gate go/no-go on internal corporate customer data** — the NC-voter run is only a public-data sanity check (pre-deduplicated; no DOB/email), so the production decision uses the org's own labelled data.

Part 3 · Line Engineers

Orientation — where the knobs are

What the code does

```
Blocker   : embed query → FAISS top-k candidates (cosine)
Linker    : Splink comparisons → per-candidate match probability
Store     : add / update / delete / save / load (no re-embed on load)
```

- Match probability = Bayesian combination of per-field weights `w = log2(m/u)`, a match prior, thresholded at `τ`.
- In-memory index today; `IndexingStrategy` / `VectorDatabase` is the seam for a vector DB later.

Scorecard to remember

- Confusion matrix (5k refs / 15k queries): **F1 99.31%**, recall 98.86%, precision 99.77%.
- Same-set blocking recall: **98.89%** @k=20 (Section 7's query set: 99.75%; compact 99.88%).
- **~7.5 ms/query amortized batched** (embed + block + one Splink run); **cold per-query online path median ~0.4 s on a quiet run** (per-query Linker construction ~0.17 s + predict ~0.19 s; can exceed ~1 s under load).
- NC-voter (real schema, synthetic mutations): **F1 92–94%**; blocking is the binding constraint at the default `k=20` (Splink becomes binding at `k=100`).

> Baseline engineering numbers — config, hardware, and data shape all move them.

Reproducing experiments

Run from repo root; scripts have `--help` and deterministic seeds (default 42).

Paper section	Script
§7 evaluation	`scripts/experiment_section7_eval.py`
§8 confusion matrix	`scripts/experiment_confusion_matrix.py`
§8.1 m/u calibration	`scripts/experiment_mu_calibration.py`
§8.2 duplicate benchmark	`scripts/experiment_duplicate_benchmark.py`
§8.3 F1 sweep	`scripts/experiment_f1_sweep.py`
calibration paradox	`scripts/experiment_mu_tau_interaction.py`
joint τ×prior surface	`scripts/experiment_mu_prior_tau_surface.py`
NC-voter replication	`scripts/ncvoter/*` + `extract_ncvoter.py`

For **your** data: population-based scripts accept `--input-records FILE`.

Gotchas — tuning & thresholds

- **Don't "just retune" m/u.** It looks worse (prior/τ coupling). Anchor the prior and re-validate prior + `τ` + weights together (Appendix — Tuning the Match Prior).
- `τ` **encodes business cost.** `τ* = C_FP/(C_FP+C_FN)`; exposed via `--threshold` / `--tau` and the F1 sweep. Raise `τ` for fewer wrong links, lower it for recall-first (Appendix — Deriving τ).
- **Tune the prior properly.** Anchor `λ`, fit with the prior pinned, tune `τ` jointly, watch the score-shift guard (Appendix — Tuning the Match Prior).

Gotchas — blocking & data

- **Blocking recall is a hard ceiling.** If recall is low, raise `k` or serialization, not the embedding model.
- **Address weakening didn't help** here — test, don't assume.
- **Real data needs a mutation/duplicate model** to score (`scripts/ncvoter/ncvoter_util.py`).
- NC voter has **no full DOB and no email** — those comparisons are weak there.

Where the details live

Whitepaper: `docs/entity_resolution_whitepaper.pdf` (source: `.tex`).

- **§3 Two-Stage** — blocking/linkage contract and costs.
- **§7 Evaluation** — recall, thresholds, latency, ablations.
- **§8, 8.1, 8.2, 8.3** — headline results and the "don't retune alone" evidence.
- **§9 NC-voter** — real schema, mutation model, `k` scaling.
- **Appendix: Deriving τ** — cost / F1 / GMM / transitivity methods.
- **Appendix: Tuning the Match Prior** — anchor `λ`, then tune `(λ, τ)` jointly (the calibration-paradox fix).
- **Use of AI** — disclosure and verification.

Checklist before you leave

1. Run `scripts/experiment_confusion_matrix.py --count 5000` and read the JSON.
2. Reproduce the paradox: `scripts/experiment_mu_tau_interaction.py --base-count 5000`.
3. Swap `--input-records <your file>` into a population-based experiment.
4. Find where **blocking recall** caps your F1 — then plan linkage work.

That is the fastest way to make these results your own.

Thank you

FAISS + Splink — one pipeline, three audiences

Bring a labelled sample of your real duplicates.